Predicting Hourly Bike Sharing Demand in Seoul Using Machine Learning

Lily Bryan
MSDS Program, CISC 5800
Fordham University
New York, NY

***Abstract*—This paper uses four different predictive modeling machine learning approaches in order to better understand hourly bicycle rental demand, using Seoul's bike sharing system dataset which runs from December 2017 to November 2018. This is done to find the best method to optimize the city's allocation of bikes. The four different methods used in this paper are Ordinary Least Squares (OLS) Regression, Lasso regularization, Logistic Regression, and Kernel Ridge Regression. In terms of preprocessing, I used one-hot encoding of the categorical variables, time-ordered training and testing splits of the data, and feature scaling. After a preliminary analysis, I identified that the Temperature and Hour features are the strongest positive predictors, and Winter and Humidity had strong negative correlations. Getting into the models themselves, I determined that Lasso regularization method did give a small improvement over the OLS baseline by eliminating the Spring season feature, but this was only an R^2 jump from the baseline of 0.34 to 0.366. Kernel Ridge Regression, however, did significantly outperform the other models using a Radial Basis Function (RBF), with an R^2 value of 0.6356 on the validation set. Ultimately, the increase in performance from the KRR method does confirm that the relationship between environmental conditions in Seoul, time, and bike demand is nonlinear. The model did show a performance decrease on the test set with an R^2 of 0.4563 (likely due to the unpredictable weather shifts in late Autumn), however these results still show that nonlinear kernel-based methods are necessary for accurately modeling these urban biking demand patterns. Future work should focus on using data over multiple years instead of just one in order to better capture the seasonal cycles, as well as integrating public events data in Seoul.**

***Index Terms*—bike sharing, demand prediction, Lasso regression, Kernel Ridge Regression, machine learning**

I. Introduction

Across the world, urban cities are introducing rental bikes as an alternative to driving and as a way to reduce CO_2 emissions. Seoul's bike sharing system is one of the largest in Asia, and serves millions of rides per year across hundreds of stations throughout the city. This system is very accessible, with over 20,000 bikes available to riders as well as over 1,500 stations in the city and operates 24/7. This enables easy transit for riders looking to go shorter distances, for example from subway stations to destinations.

However, in order for this bike sharing system to be successful, the bikes need to be available in the right stations, at the right time. Otherwise, we might see too many bikes in an area where there is little need which would waste resources, or too few bikes in an area where there is a high demand. For example, early morning commuters who might all want to bike to work would be frustrated to find not enough bikes for them, particularly if there is a surplus of bikes in another area. Thus, it is crucial to accurately predict hourly demand to ensure the system runs smoothly and efficiently.

This project will attempt to predict bicycle demand per hour using the Seoul Bike Sharing Demand dataset. While the relationship between weather and biking is intuitive, accurately predicting the numerical demand for bikes at the hourly level requires modeling the complex relationship between environmental conditions, time of day, and seasonal patterns. Thus, this paper will apply and compare multiple machine learning approaches, including linear regression, Lasso regularization, Logistic Regression, and Kernel Ridge Regression, in order to determine which method best fits the relationship between weather, time, and bike demand.

II. Dataset

The Seoul Bike Sharing Demand dataset contains counts of public bicycles rented per hour in the Seoul Bike Sharing System, with corresponding weather data, time data, and holiday information. It has 13 features and 1 target variable (rented bike count), 8,760 instances, and ranges from December 2017 to November 2018. As outlined in Table I, key features include weather descriptors such as humidity and snowfall, as well as hour of day and date, and categorical features of season, holiday, and functioning day.

The target variable, Rented Bike Count, ranges from 0 bikes to 3,556 bikes per hour, with a mean of 729.16 and a median of 542. Upon preliminary analysis, I found that Temperature has an r value of 0.56 and Hour has an r value of 0.43, while Humidity and Snowfall had negative relationships with $r = -0.20$ and -0.15 respectively. Fig. 1 shows the average bike demand by hour of day, and Fig. 2 shows demand by season. Looking at Fig. 1, we can see that there are patterns that are in line with what one would expect from typical commuting behavior, with two distinct peaks that align with morning and evening commuting times. The demand for bikes is at its lowest in the early morning hours, from about 3:00AM to 5:00AM, with about 140 to 150 bikes rented per hour. Then, there is a sharp increase in demand during the morning rush hour with the demand peaking at 8:00AM at about 1,050 bikes per hour. The demand then dips a bit during the midday hours of 10:00AM to 11:00AM (about 550 to 620 bikes per hour), and then climbs again through the afternoon hours to

the evening rush hour peak at 6:00PM (18:00) which has the highest demand quantity at 1,550 bikes per hour. The demand then gradually decreases into the evening. This graph shows that the bike sharing system is used very heavily by commuters. Fig. 2 shows us that the Summer season has the highest overall demand for bikes (∼1,035), with Autumn and Spring close behind, and Winter at a drastically lower demand point with about 225 bikes per hour. This is reasonable, as people are less likely to bike in the colder weather. These patterns are informative for preliminary examination, as they suggest that there is no one feature that will accurately predict bike demand.

TABLE I
DESCRIPTIVE STATISTICS OF DATASET FEATURES (N = 8,465)

Variable	**Type**	**Mean**	**Min**	**Max**
Target Variable				
Rented Bike Count (bikes/hr)	Num.	729.2	2	3556
Numerical Features				
Hour	Num.	11.5	0	23
Temperature (°C)	Num.	12.8	-17.8	39.4
Humidity (%)	Num.	58.2	0	98
Wind Speed (m/s)	Num.	1.7	0	7.4
Visibility (10m)	Num.	1433.9	27	2000
Dew Point (°C)	Num.	3.9	-30.6	27.2
Solar Rad. (MJ/m^2)	Num.	0.6	0	3.5
Rainfall (mm)	Num.	0.15	0	35.0
Snowfall (cm)	Num.	0.08	0	8.8
Categorical Features				
Season	Cat.	Summer 26.1%, Winter 25.5%, Spring 25.5%, Autumn 22.9%		
Holiday	Binary	No Holiday 95.2%, Holiday 4.8%		

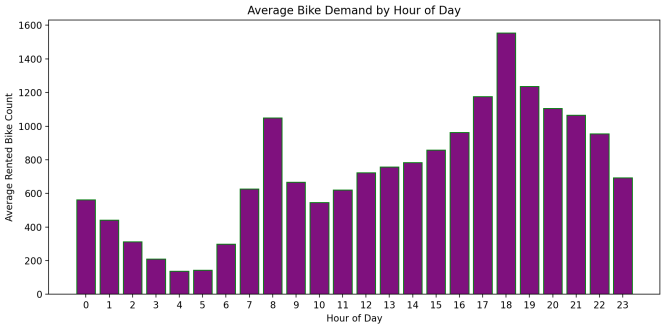

Fig. 1. Average bike demand by hour of day.

III. PREPROCESSING

In order to begin this project, I had to first fully examine and verify my data. To start the preprocessing, I used Python's Pandas library to confirm that there were no missing values in the dataset. After confirming this, I moved on to a closer inspection of the data itself, and found that the Functioning

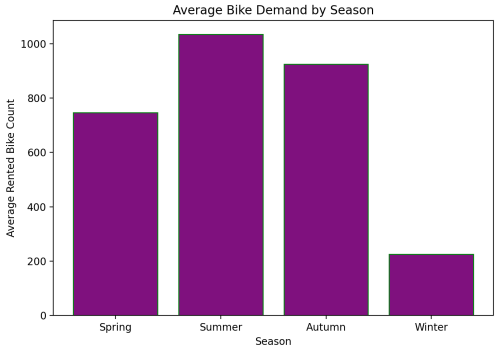

Fig. 2. Average bike demand by season.

Day category contains 295 instances of days that are non-functioning, which means that all bikes are offline and thus bike count is zero. This may be due to scheduled maintenance, or public holidays in which the system shuts down entirely. Because there are over 8,000 rows, I dropped these instances from the CSV, as the nonfunctioning days are a reflection of system downtime, and not bike demand patterns.

I then moved on to one-hot encoding of my categorical values, Season and Holiday. Using Python, one-hot encoding creates a binary column for each unique category value. Season has four values, so it becomes three columns, and Holiday has two values (yes/no) so it becomes one column. To avoid multicollinearity, one column per group was dropped. Functioning Day is also a categorical variable, however it did not need to be encoded as every row is the same (marked "yes").

Following this, the data was split into training, validation, and test sets. Without this split, the model would memorize the data, instead of learning patterns that can be applied to new and unseen data. The split provides us with data that the model has not seen, and thus can be used to evaluate its performance. 70% of the data is used as training data, which is what the model learns from. 15% of the data is used as validation, which will be used to tune the model. For example, when choosing the best regularization strength for Lasso, different values will be tried, and the one picked will be the one that performs the best on the validation set. The test set is 15% of the data, and will be used at the end of the project to report final results. The test set is kept separate from the validation set in order to avoid overfitting to the validation set through repeated tuning. Additionally, this data is split in temporal order. This is because if the data were split randomly, the training set might use data from June of 2018 while the test data uses February of 2018, which would mean that the model is trained on future data and tested on past data. Because the data is sorted in temporal order, the split was done by index, with a training size of 5,925, and validation and testing sizes of 1,270 each.

The final necessary preprocessing step for this dataset is scaling the data. This is a crucial step for this dataset in particular because the numerical features vary widely. For example,

the Visibility range is from 27 to 2000, and Snowfall is from 0 to 8.8. When we come to the Kernel Ridge Regression section of this analysis, we will see that feature scaling is a necessary step due to Kernel Ridge Regression being sensitive to the scale of its features. If we do not scale the features, the features with larger ranges will dominate over the features with smaller ranges because of their magnitude. In order to scale the data, I used the standardization approach, which transforms each feature to have a mean of 0 and standard deviation of 1. The feature scaling process was conducted using the scikit-learn package in Python (`StandardScaler`). The scaling was fit only to the training data and then applied to the validation and test data to ensure no data leakage.

IV. Linear Regression

Linear regression models the relationship between a dependent variable and independent variables by fitting a linear equation to the data. The model is in the form:

$$\hat{Y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p \tag{1}$$

where $\hat{Y}$ is the predicted bike count, β_0 is the y-intercept, $\beta_1, \beta_2, \ldots, \beta_p$ are the coefficients for each feature, and $x_1, x_2, \ldots, x_p$ are the feature values. The model is fit by minimizing the Mean Squared Error (MSE):

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2 \tag{2}$$

where Y_i is the actual bike count and $\hat{Y}_i$ is the predicted value.

The first model used is the baseline model with linear regression and model selection. To begin, I fit ordinary least squares regression with all of my features on the training data, using scikit-learn's Linear Regression package. I then evaluated the model on the validation set using Mean Squared Error (MSE) and R^2, to determine how the model does on unseen data. The MSE came out to 285,321.6 and the R^2 value came out to 0.34, which means that the model explains 34% of the variance in bike demand. Regarding the MSE, because it is in bike units it is not interpretable on its own, but will be used for comparison across models. Initially, I planned to use k-fold cross validation to estimate the model's performance, however after initial analysis it became clear this is not an option, as this method would randomly shuffle the data and thus disrupt the temporal ordering. Instead, I used scikit-learn's `TimeSeriesSplit`, which respects the temporal order of the data. The mean R^2 here came out to 0.12, which confirms that the model does struggle to consistently generalize with the baseline linear regression model. I then analyzed the coefficients to determine which features the model weights the most heavily, which are outlined below in Table II. Note that because the features are scaled, the coefficients are in terms of standard deviations as opposed to original units. As evident in the table, Temperature and Hour are the two strongest positive predictors, with coefficients of 258.42 and 172.95, respectively. The interpretation of the Temperature coefficient is that for every 1 standard deviation increase in temperature, the model predicts 258 more bikes rented per hour, and the same methodology is applied to the rest of the coefficients. Humidity has a coefficient of -179.11, which is the strongest negative predictor. It is evident that higher humidity strongly reduces predicted demand, which makes sense because high humidity is an uncomfortable biking condition. The winter season has a negative coefficient of -72.26, which means that it being winter significantly reduces the demand for bikes. The reference season here is Autumn, so this coefficient means that being in Winter versus Autumn reduces predicted demand by 72 bikes per hour. This is reasonable as people are less likely to want to bike in the colder months. Rainfall is also negative, which also makes sense as rain is not ideal biking weather. There are also a number of counterintuitive factors shown in the coefficient analysis. Snowfall has a positive coefficient of 23.83, and Solar Radiation has a negative coefficient of -53.09. These unexpected results suggest multicollinearity among the weather features. To confirm this, I performed a correlation analysis on the Winter Season feature and Temperature feature, and obtained an r value of -0.74 which indicates a strong negative relationship. This means that as the temperature decreases, the likelihood of it being Winter increases (and vice versa). This strong correlation leads to the model not being able to separate the individual effects they have on bike demand, leading to these counterintuitive coefficients. This analysis further emphasizes the need for the upcoming Lasso regularization portion of this project.

TABLE II
Linear Regression Coefficients

Feature	Coefficient
Temperature (°C)	258.42
Hour	172.95
Dew Point Temperature (°C)	100.01
Seasons_Summer	44.72
Seasons_Spring	30.97
Wind Speed (m/s)	27.77
Snowfall (cm)	23.83
Visibility (10m)	22.00
Holiday_No Holiday	19.34
Solar Radiation (MJ/m^2)	-53.09
Rainfall (mm)	-64.58
Seasons_Winter	-72.26
Humidity (%)	-179.11

V. Lasso Regularization

In order to obtain the most important coefficients (feature selection) and get rid of the irrelevant ones, the Lasso regularization technique was used:

$$\text{Lasso} = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2 + \alpha\sum_{j=1}^{p}|\beta_j| \tag{3}$$

where $\hat{Y}$ is the predicted bike count, Y_i is the actual bike count, β_j are the coefficients, p is the number of features, and α is the regularization parameter that determines how much of a penalty is applied to the coefficients.

This regularization technique applies a penalty to the absolute value of the regression coefficients, with the result of shrinking some coefficients to exactly zero, thus eliminating the irrelevant coefficients, which prevents overfitting and enhances the accuracy of the model. To perform Lasso regularization, I tried a range of alpha values, and used the validation set to pick the best one. I fit Lasso with the best alpha on the training data and then evaluated on the validation set with the MSE and R^2. I observed which coefficients were shrunk to zero, and compare my results to my baseline linear regression model. In terms of alpha values tested, I used the standard logarithmic grid values of 10^{-4} to 10^3 (0.0001, 0.001, 0.01, 0.1, 1, 10, 100, 1000).

The results of the Lasso Regularization run are shown below in Table III and Table IV. The best alpha value is 0.1, and the MSE is 275,230, which is 10,091 lower than the baseline Ordinary Least Squares value of 285,321. The R^2 value came out to 0.3663, which is a small improvement over the baseline R^2 value of 0.34. In terms of feature selection, only the Spring feature was shrunk to zero, which suggests that, actually, almost all of the features in the dataset contain meaningful information. This is an important note, as it disproves my earlier assumption that the dataset contained too many irrelevant features, and instead suggests that the relationships are nonlinear. Although Lasso can fix multicollinearity, it cannot fix nonlinearity, which explains only the slight improvement. The elimination of the Spring season does make sense, as the demand for bikes in the springtime is likely already captured in the Temperature feature, because Spring temperatures are relatively mild and correlated positively with demand.

TABLE III
LASSO ALPHA GRID SEARCH RESULTS

Alpha	MSE	R^2
0.0001	302,652.00	0.3032
0.001	302,349.85	0.3039
0.01	300,115.58	0.3091
0.1	**275,230.61**	**0.3663**
1	275,802.07	0.3650
10	281,289.29	0.3524
100	343,422.92	0.2093
1000	553,917.41	−0.2753

VI. LOGISTIC REGRESSION

The next analysis done was a Logistic Regression model:

$$P(Y = 1) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p)}} \tag{4}$$

where $P(Y = 1)$ is the probability of high demand, β_0 is the intercept, $\beta_1, \ldots, \beta_p$ are the coefficients, and $x_1, \ldots, x_p$ are the feature values. This model is a classification method that is used to predict the probability of a dependent variable in the binary form of 0 or 1, based on independent predictor variables. For this project, that means instead of predicting the exact bike quantity demand count as done in the regression model previously discussed, we predict either High or Low demand. In order to carry out this model, the median of 542 is used as the separator between high and low, so anything below 542 will be classified as Low Demand (0), and anything above 542 will be classified as High Demand (1). Then, the Logistic Regression model will be fit to the training data, and will be evaluated using accuracy, precision, recall, and F-1 score (as opposed to MSE and R^2).

TABLE IV
LASSO COEFFICIENTS (ALPHA = 0.1)

Feature	Coefficient
Temperature (°C)	262.63
Hour	172.90
Dew Point (°C)	94.76
Wind Speed (m/s)	27.59
Snowfall (cm)	23.61
Visibility (10m)	22.05
Holiday_No Holiday	19.25
Seasons_Summer	16.17
Seasons_Spring	**0.00**
Solar Radiation (MJ/m^2)	−52.84
Rainfall (mm)	−64.56
Seasons_Winter	−103.10
Humidity (%)	−176.84

The results are outlined in Table V below. The overall accuracy is 81.1%, which means that the model correctly classified demand as either High or Low 81.1% of the time, and the F1 Score is 0.8683, which means that the model has a strong balance between precision and recall for the high demand class. Taking a step further and breaking it down by class, we can see that for High Demand, the precision is 0.87 and the recall is 0.87. This means that the model is quite good at predicting high demand hours; when it predicts high demand it is correct 87% of the time, and the recall score shows it correctly catches 87% of actual demand hours. The model does not do as well with the Low demand class, with a precision of 0.66 and a recall of 0.67. The interpretation of this is that the model does not do as well with predicting the lower demand hours, and is only correct 66% of the time. There are a number of potential reasons for why the model does not perform as well with the lower class. The training set has 3,461 instances of low demand versus 2,464 instances of high demand. The low demand hours are likely more varied, as they might occur across all types of weather and time conditions, for example if it is too hot in the summer or too cold in the winter. This would make it more difficult for the model to correctly classify the low demand hours. Additionally, the validation set does have a class imbalance with 912 high demand instances and 358

low demand instances (reflective of people naturally wanting to bike more in the warmer months), so the accuracy may be inflated by the higher High demand instances. Due to this imbalance, the F-1 score and class precision and accuracy are used as the main metrics for evaluation as opposed to overall accuracy. While Logistic Regression can be a useful model for classifying the demand as either high or low, it is not useful for predicting the quantity of demand, as is the primary goal of this project, and thus leads to the next model analysis with Kernel Ridge Regression.

TABLE V
LOGISTIC REGRESSION CLASSIFICATION RESULTS

Class	Precision	Recall	F1	Support
Low Demand	0.66	0.67	0.67	358
High Demand	0.87	0.87	0.87	912
Accuracy				0.8110
Macro Avg	0.77	0.77	0.77	1270
Weighted Avg	0.81	0.81	0.81	1270

VII. KERNEL RIDGE REGRESSION

The final regression technique used in this analysis is Kernel Ridge Regression. Although Lasso Regression did slightly improve the results against the OLS baseline, the fact that it was only a small improvement does suggest that the main limitations here were not that there were irrelevant features, but more so that there is a nonlinear relationship between weather, time, and bike demand. This leads us to Kernel Ridge Regression, which is a nonlinear regression technique that combines Ridge regression with the "kernel trick." Ridge Regression, as shown in the formula below:

$$\text{Ridge} = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2 + \alpha\sum_{j=1}^{p}\beta_j^2 \tag{5}$$

is similar to Lasso regularization in that it penalizes the coefficients, but it differs because it adds a shrinkage penalty (L2 Regularization) that shrinks the coefficients towards zero but does not actually bring them down to exactly zero. By adding this penalty, Ridge regression prevents overfitting and addresses the multicollinearity that makes OLS unstable. This technique also addresses the bias-variance tradeoff, in that it reduces the variance of the estimates by introducing a small amount of bias, which leads to better prediction performance on new data. The "kernel trick" aspect of this technique is that the model learns a linear function in a high-dimensional feature space introduced by the respective kernel and the data. For nonlinear kernels, this would correspond to a nonlinear function in the original space. Instead of mapping features explicitly to their higher dimensions, this trick computes the dot product in that higher dimensional space. In terms of this project, this method allows the model to capture interactions between variables, for example how the effect of temperature on bike demand changes, depending on the hour or time of day, without needing to manually create those combined features.

This project analysis will use the Radial Basis Function (RBF) kernel. This kernel maps input data into a space with infinite dimensions, in order to enable nonlinear regression by calculating how close points are using the Euclidean distance, with similarity values ranging from 0 (not similar at all) to 1 (identical). This specific kernel was chosen for this analysis because in the RBF kernel, points that are close together will be considered similar, which works here because the relationship between weather and demand likely has smoother, more continuous patterns as opposed to sharp boundaries.

Kernel Ridge Regression has two hyperparameters that need to be tuned, alpha and gamma. The alpha hyperparameter controls the regularization strength (as with Lasso regularization), so a higher alpha means a higher penalty to the coefficients. The Gamma parameter controls the width of the RBF kernel, so it determines how quickly the similarity between two points goes away as their distance increases. So if the Gamma is smaller, then points that are far away can still be classified as similar, and if the Gamma is bigger, then only points that are quite close to each other can be considered similar. In order to obtain these values, I performed a grid search, which means trying different combinations of alpha and gamma values and picking the pair that gives us the smallest MSE on the validation set. The Alpha values tried were 0.001, 0.01, 0.1, 1, 10, 100 and the Gamma values tried were 0.0001, 0.001, 0.01, 1.

The results of this analysis are below in Table VI. The best performing alpha was 0.1 and the best performing gamma was 0.1 as well. The lowest MSE found is 158,281 and the R^2 value is 0.6356. This is a very significant improvement over the baseline R^2 value (0.34) and the Lasso Regularization R^2 (0.3663). This large improvement jump confirms that the relationship between weather, time, and bike demand is actually not linear, as I had previously assumed. The linear models, even with regularization, could not capture this complex relationship. However, by mapping the features onto a higher dimensional space using the RBF kernel, Kernel Ridge Regression was able to model these nonlinear patterns and significantly improve the accuracy of bike demand quantity predictions.

TABLE VI
KERNEL RIDGE REGRESSION BEST RESULT

Alpha	Gamma	MSE	R^2
0.1	**0.1**	**158,281.68**	**0.6356**

VIII. COMPARISON AND OVERALL RESULTS

This section will compare and analyze all models. The models were applied in order of increasing complexity, as shown in Table VII below, in order to determine whether more complexity leads to better and more accurate predictability results. Note, as previously stated, the regression models are compared using MSE and R^2, while the Logistic Regression model is compared using classification metrics as it answers

a different question. Additionally, the final results on the test set will be analyzed for the best performing model.

In terms of the Regression models used, this project started with Ordinary Least Squares, then used Lasso Regression, then Kernel Ridge Regression with the RBF kernel. Lasso improved results slightly by addressing the multicollinearity that was present as well as removing the Spring season feature, but it was only a slight increase in R^2, from the baseline OLS of 0.34 to 0.3663 which is about a 3.5% increase. This led to the question of whether or not nonlinearity was the real issue, and it was confirmed to be so when the KRR analysis was conducted, as the R^2 value jumped to 0.6356, which is nearly double the original R^2 value. The MSE for KRR also improved significantly against both the baseline model of 285,321 and the Lasso Regression model of 275,230 to 158,281. Additionally, the multicollinearity finding of the Winter season feature and Temperature feature having an r value of -0.74, and the elimination of the Spring feature points towards nonlinearity being the main problem, instead of the initial assumption of it being irrelevant features. This suggests that predicting demand for this bike sharing system is not as simple as assuming each feature has an individual and linear contribution, and actually shows that a model that can capture complicated relationships between features is necessary.

In terms of the Logistic Regression analysis, this answered a slightly different question. Instead of looking at the quantity of bikes predicted, it looked to classify high versus low demand. Due to the class imbalance in the dataset (912 high demand versus 358 low demand instances), overall accuracy was not used as the primary metric. Instead, this analysis focuses on per-class F1, precision, and recall. Overall, the model did well with the precision and recall for the High demand class, with both at 0.87; however the Low demand class was harder to predict, with a precision of 0.66 and recall of 0.67. This is likely due to the lower demand hours being spread across all seasons and times, so it is more difficult to consistently classify. The fact that the Logistic Regression model struggles to predict for the Low class further emphasizes the finding that the demand patterns are complicated and not captured well by only using linear boundaries.

Finally, the best performing model, Kernel Ridge Regression, is applied to the test set. All of the previous models were applied to the validation set, and the test set was saved for this final evaluation in order to get an unbiased estimate of the model's performance. After running this model, the test set R^2 value came out to 0.4563 and an MSE of 166,003. This test set R^2 value being lower than the validation set can be attributed to the temporal aspect of the dataset. The training and validation sets cover Winter, Spring, Summer, and early Autumn, while the test set covers late Autumn from October to November 2018. These months do have quickly changing weather conditions that are inherently more unpredictable than the late summer and early autumn months that are covered by the validation set. The slight increase in MSE of about 4.9%, however, does suggest that the model generalized reasonably well despite the temporal order challenge.

TABLE VII
REGRESSION MODEL COMPARISON

Model	Set	MSE	R^2
Linear Regression (OLS)	Validation	285,321.6	0.3400
Lasso (α = 0.1)	Validation	275,230.6	0.3663
KRR (α = 0.1, γ = 0.1)	Validation	158,281.7	0.6356
KRR (α = 0.1, γ = 0.1)	**Test**	**166,003.0**	**0.4563**

IX. CONCLUSION

This project aimed to predict hourly bicycle rental demand in Seoul's bike sharing system by applying and analyzing different machine learning models: linear regression, Lasso regularization, Logistic Regression, and Kernel Ridge Regression. This was done in order to determine the best method to capture the complex relationship between weather, time, and bike demand. Kernel Ridge Regression with the RBF kernel was found to be the best model to capture this relationship. This improvement of KRR over the linear models confirmed that the relationships between weather, time, and bike demand are nonlinear, so the model needs to be one that is capable of capturing more complex feature interactions.

There are a number of limitations in this project. The most significant limitation is that the model was trained and validated mainly on Spring, Summer, and early Autumn data, but tested on late Autumn to Winter data. What this means is that the model barely sees the weather patterns of October to November, which again, may explain the decrease in R^2 value on the test set. If the dataset had more years in it, it would be able to see these late Autumn patterns during the training portion and likely be able to better generalize. Seeing as the R^2 value is 0.45 with KRR (so 45% of the variance of the test set can be explained), it is clear that there are outside factors that are influencing bike demand. Some examples of these factors might be holidays, special events in the city, construction near bike stations, etc.

In terms of future work, there are multiple avenues I would be interested in exploring. The most impactful future work that can be obtained is more than one year of data, as explained in the limitations paragraph above; having this data would allow for the model to be exposed to repeated seasonal cycles. Additional features would also be useful to include in the dataset. For example, public transit data or Seoul event calendars would allow us to better understand what drives demand outside of the given dataset. Another option would be to utilize a Neural Network, because this could allow the model to learn more complicated relationships between the features, such as rain by rush hour. Seeing as KRR did confirm nonlinearity, this approach might allow for an improvement in test set performance. A final option for future work might be datasets for each station in Seoul, because this would enable us to predict demand at individual stations.

REFERENCES

[1] "Seoul Bike Sharing Demand," UCI Machine Learning Repository, 2020. [Online]. Available: https://doi.org/10.24432/C5F62R